In [2]:
```
# Q1: Marks Analysis with Conditions

import pandas as pd

marks = [45, 67, 89, 34, 90, 55, 72]
subjects = ['Math', 'Physics', 'Chemistry', 'English', 'CS', 'Biology', 'History']

s = pd.Series(marks, index=subjects, name="Marks")

# Vectorized filtering
high_scores = s.loc[s.gt(70)]

# Safe labeling (no dtype conflict)
result = s.to_frame()
result["Status"] = pd.cut(
    s,
    bins=[-1, 39, 100],
    labels=["Fail", "Pass"]
)

# Above average count
above_avg_count = s.gt(s.mean()).sum()

print(high_scores)
print(result)
print("Above Avg Count:", above_avg_count)
```

```
Chemistry    89
CS           90
History      72
Name: Marks, dtype: int64
           Marks Status
Math          45   Pass
Physics       67   Pass
Chemistry     89   Pass
English       34   Fail
CS            90   Pass
Biology       55   Pass
History       72   Pass
Above Avg Count: 4
```

In [4]:
```
# Q2: Salary Increment Logic

import pandas as pd

salary = [25000, 32000, 28000, 40000, 50000]
emp = ['A', 'B', 'C', 'D', 'E']

s = pd.Series(salary, index=emp, name="Salary").astype(float)

# Apply transformations
updated_salary = s.mask(s < 30000, s * 1.10)
updated_salary = updated_salary.mask(updated_salary > 45000, updated_salary * 0.95)
```

```
print(updated_salary)
print("Total Expense:", updated_salary.sum())
```

```
A    27500.0
B    32000.0
C    30800.0
D    40000.0
E    47500.0
Name: Salary, dtype: float64
Total Expense: 177800.0
```

In [5]:

```
# Q3: Missing Data Handling

data = [10, None, 30, None, 50, 60, None]
s = pd.Series(data, name="Values")

# Audit missing
missing_before = s.isna().sum()

# Impute using mean
s_filled = s.fillna(s.mean())

# Validation
missing_after = s_filled.isna().sum()

# Final clean dataset
s_clean = s_filled.dropna()

print(f"Missing Before: {missing_before}, After: {missing_after}")
print(s_clean.describe())  # Summary stats
```

```
Missing Before: 3, After: 0
count     7.000000
mean     37.500000
std      15.679073
min      10.000000
25%      33.750000
50%      37.500000
75%      43.750000
max      60.000000
Name: Values, dtype: float64
```

In [6]:

```
# Q4: Student Grades System

marks = [95, 82, 67, 58, 73, 49, 88]
students = ['S1','S2','S3','S4','S5','S6','S7']

s = pd.Series(marks, index=students, name="Marks")

# Vectorized grading
grades = pd.cut(
    s,
    bins=[-1, 59, 74, 89, 100],
    labels=["D", "C", "B", "A"]
)

# Convert to ordered categorical
```

```
grades = grades.astype("category")

grade_distribution = grades.value_counts().sort_index()

print(grades)
print(grade_distribution)
```

```
S1    A
S2    B
S3    C
S4    D
S5    C
S6    D
S7    B
Name: Marks, dtype: category
Categories (4, object): ['D' < 'C' < 'B' < 'A']
Marks
D    2
C    2
B    2
A    1
Name: count, dtype: int64
```

In [7]:

```
# Q5: Series Alignment

s1 = pd.Series([10, 20, 30], index=['A', 'B', 'C'], name="S1")
s2 = pd.Series([5, 15, 25], index=['B', 'C', 'D'], name="S2")

# Default alignment (NaN expected)
raw_sum = s1 + s2

# Robust addition (handles missing keys)
aligned_sum = s1.add(s2, fill_value=0)

print(raw_sum)
print(aligned_sum)
```

```
A     NaN
B    25.0
C    45.0
D     NaN
dtype: float64
A    10.0
B    25.0
C    45.0
D    25.0
dtype: float64
```

In [8]:

```
# Q6: Ranking System

scores = [88, 92, 79, 93, 85]
players = ['P1','P2','P3','P4','P5']

s = pd.Series(scores, index=players, name="Scores")

# Dense ranking (no gaps)
rank = s.rank(method="dense", ascending=False)
```

```
# Insights
top_3 = s.nlargest(3)
lowest_player = s.idxmin()

summary = pd.DataFrame({
    "Score": s,
    "Rank": rank
}).sort_values("Rank")

print(summary)
print("Top 3:\n", top_3)
print("Lowest:", lowest_player)
```

```
    Score  Rank
P4     93   1.0
P2     92   2.0
P1     88   3.0
P5     85   4.0
P3     79   5.0
Top 3:
 P4    93
P2    92
P1    88
Name: Scores, dtype: int64
Lowest: P3
```

In [9]:

```
# Q7: String Operations

names = ['anuj', 'rahul', 'sneha', 'kiran', 'amit']
s = pd.Series(names, name="Names")

# Clean + transform
names_cleaned = (
    s.str.strip()
     .str.title()
)

# Pattern filtering
contains_a = names_cleaned[names_cleaned.str.contains("a", case=False)]

# Replace with mapping
updated_names = names_cleaned.replace({"Anuj": "Anuj Kumar"})

print(names_cleaned)
print(contains_a)
print(updated_names)
```

```
0     Anuj
1    Rahul
2    Sneha
3    Kiran
4     Amit
Name: Names, dtype: object
0     Anuj
1    Rahul
2    Sneha
3    Kiran
4     Amit
Name: Names, dtype: object
0    Anuj Kumar
1         Rahul
2         Sneha
3         Kiran
4          Amit
Name: Names, dtype: object
```

In [10]:

```
# Q8: Chained Operations

data = [5, None, 15, 25, None, 35, 45]

result = (
    pd.Series(data, name="Data")
    .pipe(lambda x: x.fillna(x.median()))
    .loc[lambda x: x > 20]
    .sort_values(ascending=False)
    .reset_index(drop=True)
)

print(result)
```

```
0    45.0
1    35.0
2    25.0
3    25.0
4    25.0
Name: Data, dtype: float64
```

In [11]:

```
# Q9: Sales Performance

sales = [200, 450, 300, 150, 500, 700, 100]
days = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']

s = pd.Series(sales, index=days, name="Sales")

# KPIs
best_day = s.idxmax()
worst_day = s.idxmin()
total_sales = s.sum()
avg_sales = s.mean()

# Bonus logic
sales_with_bonus = s.where(s <= 400, s * 1.20)
```

```python
# Ranking
ranked_sales = s.sort_values(ascending=False)

# Summary dashboard
summary = pd.DataFrame({
    "Sales": s,
    "With Bonus": sales_with_bonus
})

print(summary)
print(f"Best: {best_day}, Worst: {worst_day}")
print(f"Total: {total_sales}, Avg: {avg_sales}")
print(ranked_sales)
```

```
     Sales  With Bonus
Mon    200         200
Tue    450         540
Wed    300         300
Thu    150         150
Fri    500         600
Sat    700         840
Sun    100         100
Best: Sat, Worst: Sun
Total: 2400, Avg: 342.85714285714283
Sat    700
Fri    500
Tue    450
Wed    300
Mon    200
Thu    150
Sun    100
Name: Sales, dtype: int64
```